LedgerDirect for PrestaShop



Accept XRP, RLUSD and USDC directly on the XRP Ledger — no payment processor, no custody, funds land in the merchant's own wallet.

The module is the PrestaShop 9 adapter over `hardcastle/ledger-direct-core`, the shared package that holds the XRPL and pricing logic. Everything platform-specific — persistence, checkout, order states, the settings screen — lives here; price conversion, the oracle set, the stablecoin registry and ledger sync live in the core and are not reimplemented.

How it works

A customer picks XRP, RLUSD or USDC at checkout. The order is placed straight away in a **Awaiting XRPL payment** state and the customer gets a payment page: the exact amount, the shop's receiving address, and a **destination tag** unique to that order. The tag is what ties an incoming ledger transaction back to the order, so one wallet address serves every customer.

The quote is fixed for a configurable window (five minutes by default). Reloading the page never changes the amount; once the quote lapses the customer can ask for an updated one, and the destination tag stays the same so a payment already in flight is not orphaned.

Payments are confirmed three ways, which is deliberate redundancy:

- the payment page polls while the customer is watching,
- a **check now** button for anyone who would rather not wait (and for browsers without JavaScript),
- a cron endpoint that settles orders for customers who closed the page.

An order is credited only from the ledger's `delivered_amount`, only when what arrived covers the requested amount, and — for stablecoins — only when the currency and issuer match. Several payments in the quoted asset add up, so a customer who sent too little can send the rest. Anything else stays open, and the payment page says why: it shows one of five states — waiting, expired, partial payment (with the outstanding amount), wrong token (with the full amount still due), or paid — and updates in place while the customer watches, without reloading.

The merchant sees the same thing from the other side. A payment that arrives but does not pay the order moves it to its own state, **XRPL payment incomplete** — visible in the order list, filterable, and dated in the order history — and the order page carries a LedgerDirect panel with the state, what was requested, what arrived, what is still outstanding, and every transaction on the order's destination tag linked to the explorer. Nothing on that panel changes the order; settling remains the sync's job.

The page asks the server every 8 seconds. That endpoint syncs with the XRPL node at most once every 5 seconds per receiving account, whatever the number of customers waiting; in between it answers from what is already stored. A guest order can poll too — the link carries the order's secret, and no login is required.

Requirements

- PrestaShop 9
- PHP 8.2 or newer
- An XRP Ledger account. Accepting RLUSD or USDC additionally requires a **trustline** to the respective issuer on that account, or payments will fail on the ledger.

Installation

Install the module from the Back Office (Modules → Module Manager → Upload a module) or drop this directory into `modules/` as `ledgerdirect`.

Installing creates two tables for ledger data plus one for payment records, and registers the "Awaiting XRPL payment" order state. Uninstalling deliberately **keeps** the tables: the destination-tag counter cannot be reconstructed, and reusing tags would match new orders against old payments.

Upgrading from an earlier version runs the module's migrations (`upgrade/`) through PrestaShop's usual upgrade step in the Module Manager. Version 0.2.0 adds a `network` column to the synced transactions; existing rows are backfilled from their CTID. Version 0.5.2 renames the CTID column to `tx_ctid`, the name core 0.9 uses on every platform.

PrestaShop does not run that step by itself when newer files arrive. Until it has run, the module's configuration page and the LedgerDirect panel on each order name both versions, and a failed sync says in the log (Advanced Parameters → Logs) that the upgrade is pending — payments may not be confirmed while the database is behind the files.

Configuration

Modules → LedgerDirect → Configure:

Setting	
Receiving address	The shop's XRPL account. Validated as an XRPL address before it is saved.
Network	`testnet` or `mainnet`. The address must belong to the selected one.
Payment methods	XRP, RLUSD, USDC. Stablecoins are off by default — they need a trustline first.
Quote validity	How long a quoted amount stays fixed, 60–3600 seconds.

Setting	
Logo	What the payment page shows in its header: the shop logo, a picture from the shop's `img/` directory (a path relative to it, e.g. `brand/logo.png`), or the first letter of the shop name.
Accent colour	The one colour of the payment page — buttons, countdown bar, destination tag — with white text on it, so it must be dark enough (contrast 4.5:1). Default `#1f5eff`.
Xaman API key, WalletConnect project id	Public identifiers, optional. With one of them, customers on a phone get an "Open in wallet app" button.

Issuer addresses are shown on that screen but cannot be edited. They are fixed in the core: a wrong issuer would send customer funds to a dead trustline.

The same screen shows the cron URL, including its token:

```
https://<shop>/module/ledgerdirect/cron?token=<token>
```

Call it every few minutes. Without it, a customer who closes the payment page before their transaction confirms is only settled the next time someone visits their payment page.

The payment page

After checkout the customer lands on `/module/ledgerdirect/payment`, reachable by the order's `secure_key`, so a guest can return to it. The page stands on its own — the module answers PrestaShop's `overrideLayoutTemplate` hook with the theme's content-only layout for this one page, so the theme's `<head>` and stylesheets stay and its header, footer and columns go.

Behaviour and design come from `@ledger-direct/payment-ui`, the package every LedgerDirect plugin shares: the five payment states, the countdown, polling, copy buttons, the QR code with address, destination tag and amount, and browser wallets over XRPL Connect (Crossmark, GemWallet, MetaMask Snap, Ledger, Otsu, Xyra; Xaman and WalletConnect with the identifiers above). The module has no build step: it copies the package's built files into `views/`.

File	From the package	Loaded
`views/css/payment-page.css`	`dist/payment-page.css`	with the page
`views/js/ledger-direct-payment-ui/payment-page.js`	`dist/payment-page.js`	with the page
`views/js/ledger-direct-payment-ui/wallets.js`	`dist/wallets.js`	only when the customer opens the wallet list, by a native `import()`

`views/js/ledger-direct-payment-ui/VERSION` names the package tag the files were copied from. To update: copy the three files from the package's `dist/` at the new tag and write the tag into `VERSION`. The markup the template renders is the package's contract (`src/README.md` there); every sentence a customer reads is in `views/templates/front/payment.tpl` and the XLIFF catalogues, nothing is rounded or reformatted in the browser.

Without JavaScript the page still works: the amount, address and tag are server-rendered, the QR code is drawn on the server from the same payment request, and the "Check payment now" button is a plain form.

Development

How the whole of LedgerDirect is tested across the core, the shared page package and the four plugins — the layers, what each catches, the nightly end-to-end runs and the manual cases — is in `docs/testing.md` of the core.

Development tooling lives in `dev/`, as a Composer project of its own. That separation is load-bearing: PrestaShop autoloads the module's `vendor/` on every request, so a dev dependency installed there can shadow the shop's own copy of the same package. `dev/vendor/` is never autoloaded.

The shop runs from a Docker Compose harness that sits one level above this repository and mounts this directory into `modules/ledgerdirect`.

```
composer install                # the module's runtime dependencies
composer install -d dev         # test tooling

docker compose up -d            # from the harness directory; :8080, Back Office at
/admin-dev
```

```
composer test -d dev            # unit suite, no PrestaShop needed

docker compose exec -u www-data -w /var/www/html/modules/ledgerdirect prestashop \
  php dev/vendor/bin/phpunit -c dev/phpunit.xml.dist --testsuite integration

docker compose exec -u www-data -w /var/www/html/modules/ledgerdirect prestashop \
  php dev/vendor/bin/phpstan analyse -c dev/phpstan.neon.dist
```

The unit suite runs anywhere. The integration suite needs a booted PrestaShop and its database, and is offline by design — it builds payment records directly rather than calling live price oracles.

The code follows the PrestaShop coding standard:

```
composer cs-check -d dev        # report violations
composer cs-fix -d dev          # fix them
```

Continuous integration

Every push and pull request runs `.github/workflows/ci.yml`, modelled on the checks PrestaShop applies to its own modules: PHP syntax on 8.2–8.4, PHP-CS-Fixer, PHPStan against PrestaShop 9.0 and 9.1, both PHPUnit suites, and a release archive. The integration suite runs in PrestaShop Flashlight, a pre-installed shop that boots from a dump; the same stack can be started locally with `docker compose -f dev/ci/docker-compose.yml up -d --wait` (shop on :8000).

The release job builds the zip a merchant uploads: `git archive` applies the `export-ignore` rules in `.gitattributes`, runtime dependencies are bundled with `composer install --no-dev`, and every directory gets an `index.php`. The zip is attached to the workflow run as an artifact.

Translations

English, German, French and Spanish, as XLIFF catalogues under `translations/<locale>/`.

License

MIT — see LICENSE.md.